

REGTR+

Honours Thesis Milestone Seminar · Speaker Script

Target: 22 min talk + Q&A · Read aloud — large text, 1.5 line spacing

HOW TO USE THIS SCRIPT

Each slide has three blocks. **ON SLIDE** = what the audience sees. **SAY** = read this aloud (large serif text). **DELIVERY** = the red-bar note is what to do with your body and voice. Don't read the delivery notes out loud.

QUICK REFERENCE — GLANCE BEFORE YOU START

CORE CLAIM in one sentence:

REGTR+ extends RegTR with two drop-in modules — colour-fused features (CEFE) and rigid-invariant relational positional encoding (GeoTransformer-style Q·R) — both end-to-end, both toggleable, both already implemented.

PEOPLE & PROJECT

Supervisors: A/Prof. Bruce Lu · Dr. Sheldon Feng

Baseline: RegTR (2022, end-to-end transformer registration, no RANSAC)

Comparison target: ColorPCR (2024, colour-aware but heavier multi-stage)

Datasets: 3DMatch (geometry), Color3DMatch / Color3DLoMatch (colour + geometry)

KEY NUMBERS YOU MUST KNOW COLD

Rigid-invariance tolerance	Empirically < 1e-3 on the relational tensor (NOT 5e-5). 80 random points, random SE(3). Test: src/tests/test_relational_pos_emb.py
Colour representation	Per-point RGB, normalised to [0,1] then zero-centered to [-0.5, +0.5]. NO HSV. Fed straight into KPConv.
Baseline input feature	<code>torch.ones_like(pts0[:, 0:1])</code> — a constant feature column of shape (N,1). Say 'a constant feature column,' not 'all ones.'
Hardware	VastAI 2× ~16 GB GPU pod (NOT RTX 5090 / 32 GB).
Batch size	<code>train_batch_size</code> : 1 for ALL Color3DMatch ablation presets, so gradient noise matches across the four cells.
Training	6 transformer layers, <code>d_embed</code> =256, first voxel size 0.025 m, AdamW <code>lr</code> =1e-4, gradient clipping 0.1.
Hybrid PE (default)	<code>conf/color3dmatch.yaml</code> : <code>transformer_encoder_has_pos_emb</code> : True AND <code>use_relational_pos_emb</code> : True. Sine on absolute coords + relational Q·R on top.
Pure relational ablation	<code>conf/color3dmatch_relpos_pure.yaml</code> — for the strictest invariance ablation.
Relational sigma_d	1.0 in Color3DMatch configs (NOT 0.2). Bottleneck point spacing ~0.2 m after strided KPConv, so 0.2 made the kernel vanish; 1.0 spreads signal.
Relational k for triplet angles	3-NN. From GeoTransformer.
Test invariance test	<code>test_rigid_invariance()</code> — 80 random points, random SE(3), max per-pair diff < 1e-3 (float32 round-off + k-NN tie-breaking).

CONFIG TOGGLES — say these EXACTLY:

use_color · color_source (auto / random / zeros) · use_relational_pos_emb
 transformer_encoder_has_pos_emb · sa_val_has_pos_emb · ca_val_has_pos_emb · pos_emb_type
 relational_hidden_dim (128) · relational_sigma_d · relational_sigma_a · relational_angle_k (3)

FOUR-WAY ABLATION PRESETS (Color3DMatch):

color3dmatch_baseline.yaml → geometry on coloured data → regtr_baseline_color3dmatch
 color3dmatch_color.yaml → colour only → regtr_color_color3dmatch
 color3dmatch_repos.yaml → relational only → regtr_repos_color3dmatch
 color3dmatch.yaml → full REGTR+ hybrid PE → regtr_plus_color_relational
 color3dmatch_repos_pure.yaml → pure relational + colour → regtr_repos_pure_color3dmatch (optional ablation)
 Upstream baseline name: regtr_regressCoor (in conf/3dmatch.yaml — plain 3DMatch, no modules)

TIMING MAP — finish around 22:00 to leave 3 min for Q&A

#	Slide	Target	Running
1	Title	0:30	0:30
2	What is registration	2:00	2:30
3	Why it's hard	2:00	4:30
4	Brief history	2:00	6:30
5	The gap (4-row table)	1:30	8:00
6	REGTR+ design philosophy	1:30	9:30
7	Architecture diagram	3:00	12:30
8	Task 1 — CEFE deep-dive	2:00	14:30
9	Task 2 — relational PE deep-dive	2:00	16:30
10	Implementation status	1:30	18:00
11	Preliminary results	2:00	20:00
12	Qualitative example	1:30	21:30
13	Roadmap	1:00	22:30
14	Summary	0:30	23:00

If your run-through goes over 25 min, cut: (a) slide 4 history — drop from six methods to four, and (b) slide 10 engineering — skim, don't dwell.

SLIDE 1 — TITLE

[0:30]

ON SLIDE

- Title: REGTR+: Colour-Fused, Geometry-Aware Point Cloud Registration with Relational Positional Encoding
- Your name, supervisors, 'Honours Thesis Seminar · April 2026'
- One hero image (aligned registration result)

SAY

Good morning. I'm James, supervised by A/Prof. Bruce Lu and Dr. Sheldon Feng. Today I'll talk about a deep-learning method for aligning 3D scans — point cloud registration — and specifically about teaching an end-to-end transformer model to use

colour and rotation-invariant geometry at the same time. I'll cover the problem, my two implemented contributions, my preliminary training results, and what's left on the roadmap.

DELIVERY *Stand still, deep breath before starting. Don't rush the opening — the first 15 seconds set your credibility.*

SLIDE 2 — WHAT IS POINT CLOUD REGISTRATION?

[2:00]

ON SLIDE

- Three panels: Scan A, Scan B, Aligned
- Goal: find the rigid transform (R, t) that maps Scan B into Scan A's coordinate frame
- Applications: 3D scene reconstruction, LiDAR SLAM in autonomous driving, AR/VR

SAY

A point cloud is just an unordered set of 3D points — usually from a depth camera, LiDAR, or multi-view 3D reconstruction. The registration problem is this: you take two scans of the same scene from different viewpoints — call them Scan A and Scan B — and you want to find the rigid transform, a rotation and a translation, that maps Scan B into Scan A's coordinate frame.

Once they're aligned, you can fuse them into a single reconstruction. This is the core operation behind multi-view 3D scanning, loop closure in LiDAR SLAM for autonomous driving, and AR systems that need to know where the device is in a previously-seen space. Every downstream 3D task depends on getting this right.

DELIVERY Slow down on 'rigid transform — rotation and translation.' This is the one piece of jargon non-experts need. Point at the panels as you talk.

SLIDE 3 — WHY IS REGISTRATION HARD?

[2:00]

ON SLIDE

- Three stacked callouts: Low overlap / Repetitive geometry / No colour signal
- Visual: two fragments with an ambiguous bare-wall region highlighted

SAY

Three things make this hard in practice. First, low overlap. 'Overlap' means the fraction of points that are visible in both scans. When overlap drops below about 30%, there are simply too few shared points for geometry-only matching to work reliably.

Second, repetitive geometry. Indoor scenes are full of flat walls, planar floors, and symmetric furniture — these produce many geometrically plausible but wrong matches. A bare white wall looks the same from a thousand angles.

Third — and this is the thread my work pulls on — colour can disambiguate where geometry cannot. A painted wall and a bare concrete wall have identical geometry but different RGB. Current state-of-the-art end-to-end registration models **throw this signal away entirely**.

DELIVERY Hit 'throw this signal away entirely' firmly — this is the motivation for Task 1 and you want it to land.

SLIDE 4 — A BRIEF HISTORY

[2:00]

ON SLIDE

- Timeline: ICP (1992) → FPFH (2009) → DCP (2019) → Predator (2021) → REGTR (2022) → ColorPCR (2024)
- Tags below: geometric / handcrafted / learned / overlap-aware / end-to-end / colour-aware

SAY

Thirty years of progress, in one line. ICP — iterative closest point — was the original geometry-only algorithm and still underlies most classical pipelines. FPFH replaced hand-tuned iteration with handcrafted local descriptors. DCP in 2019 was the first to learn those descriptors end-to-end with a deep network.

Predator in 2021 added overlap prediction, so the model could focus on the parts of the scans that were actually shared. REGTR in 2022 made the whole pipeline end-to-end with transformers — no RANSAC, no iterative refinement, just one forward pass — and it's the baseline I build on. ColorPCR in 2024 was the first to bring colour into a learned registration pipeline, but it's a heavier, multi-stage architecture.

The gap is visible on one line: REGTR is fast and end-to-end but geometry-only. ColorPCR uses colour but is slow and complex. My project — REGTR+ — combines both.

DELIVERY You have 2 minutes for six methods — that's 20 seconds each. Don't linger. Practice saying the name + one tag ('DCP — first learned descriptors') without hesitating.

SLIDE 5 — THE GAP

[1:30]

ON SLIDE

- 4-row table: Limitation in RegTR | Inspiration | REGTR+ Fix | Status
- Row 1: No colour | ColorPCR CEFE | Fuse RGB into KPConv backbone | ✓ Done
- Row 2: Rotation-sensitive PE | GeoTransformer | Q·R relational + hybrid sine on Color3DMatch | ✓ Done
- Row 3: Tokens attended equally | AOF (planned) | Overlap-gated attention, last K layers | Planned
- Row 4: Uniform corr weights | Kendall & Gal 2017 | σ head, $w = \hat{\sigma}/\sigma^2$ | Planned

SAY

Here's the gap, laid out against four specific limitations of the REGTR baseline. Each row is: the limitation, where I borrow the fix from, what I actually implement, and whether it's done. Rows one and two are implemented and are what I'll spend most of this talk on.

No colour — fix inspired by ColorPCR's CEFE block, but dramatically simplified down to a single-level input injection. Rotation-sensitive positional encoding — fix inspired by GeoTransformer: a relational structure embedding enters self-attention in the Q·R form used in GeoTransformer, built from pairwise distances and local triplet angles. On Color3DMatch my default preset keeps absolute sine PE as well and adds relational on top, because pure relational-only was a cold start; a separate YAML turns sine off if you want that ablation.

Rows three and four are planned for the remainder of the project: active overlap focusing to gate attention, and uncertainty-weighted SVD for the pose solver. I'll come back to these on the roadmap slide.

DELIVERY Walk the audience's eye top-to-bottom with your cursor. Don't read the table — narrate it.

SLIDE 6 — DESIGN PHILOSOPHY

[1:30]

ON SLIDE

- Top line: 'Four targeted, drop-in improvements — each independently toggleable for clean ablation.'
- 2x2 grid of the four tasks (CEFE, Relational PE, AOF, Uncertainty-weighted SVD)

SAY

Before the architecture, one design principle I want to emphasise, because it's central to how this project can be evaluated. Every one of these four changes is a drop-in, config-toggleable module. I don't rewrite the RegTR architecture — I add components that can be switched on or off with a boolean in a YAML file. That gives clean ablation for free, and makes it honest to attribute any result to a specific module rather than a soup of changes.

Two tasks are implemented — Task 1 is colour-enhanced feature extraction and Task 2 is relational structure encoding in self-attention, GeoTransformer-style Q·R, with a hybrid sine + relational preset on Color3DMatch. The other two — active overlap focusing and uncertainty-weighted SVD — are in active development. All four share the same pattern: minimal surgery, orthogonal contributions, clean ablation surface.

DELIVERY *This is your 'methodological maturity' moment — examiners love the word ablation used correctly. Don't skip it.*

SLIDE 7 — ARCHITECTURE DIAGRAM (CENTREPIECE)**[3:00]****ON SLIDE**

- Full diagram: 2 inputs → KPConv encoders (shared) → self-attn ×3 → cross-attn ×3 → corr decoder → weighted SVD
- Green callout 1 (input): 'Task 1: per-point RGB replaces constant features → use_color: true'
- Green callout 2 (self-attn): 'Task 2: GeoTransformer-style Q·R relational term (+ optional sine PE hybrid) → use_relational_pos_emb: true'

SAY

This is the centrepiece of the talk. I'll walk you through it left to right, then come back for the two green boxes.

Two input point clouds, A and B. Each goes through the same KPConv encoder — kernel-point convolutions, shared weights — which does hierarchical subsampling down four levels and produces a compact set of feature tokens for each cloud. Those tokens enter a transformer encoder: first three self-attention layers per cloud, then three cross-attention layers between clouds. The cross-attention is where the model learns which source token corresponds to which target token. Finally, a correspondence decoder produces per-point matches with confidence scores, and a weighted SVD on top solves for the rigid transform.

Now the two green boxes. At the input — the first green callout — this is Task 1. The baseline RegTR feeds a constant-1 feature into the encoder, so the encoder only sees geometry. I replace that with per-point RGB, so colour and geometry get fused through the KPConv hierarchy.

In the self-attention block — the second green callout — this is Task 2. Stock RegTR uses sinusoidal encoding on absolute coordinates, which is not rotation-invariant. I add a relational structure term in the GeoTransformer Q·R style — pairwise distances and local triplet angles — so that piece is rigid-invariant. On my main Color3DMatch config I actually keep the sine PE and add relational on top — a hybrid — because it trains more reliably than jumping straight to relational-only; the invariance story still applies to the relational component, and the test suite checks that component in isolation. Everything else is stock RegTR.

DELIVERY *Point at each block as you name it. This is your longest continuous speaking block — breathe between sentences. If you run long here, cut the 'hierarchical subsampling down four levels' sentence — the audience doesn't need it.*

SLIDE 8 — TASK 1 DEEP-DIVE — CEFE

[2:00]

ON SLIDE

- Top comparison: Baseline `feats0 = ones(N,1)` vs REGTR+ `feats0 = stack([R,G,B]) - 0.5`
- Caption: 'Each point carries its RGB into the KPConv hierarchy, zero-centered for stability'
- Config block: `in_feats_dim 1→3, use_color: true, color_source: auto (alts: random, zeros)`

SAY

Task 1 is actually the smaller of the two changes, surgically. Baseline RegTR constructs its input feature tensor as a constant-1 column — one scalar per point, always the same value. The KPConv encoder then sees only geometry. My change replaces that constant with a per-point RGB triple, normalised to the zero-to-one range and then zero-centered. No HSV conversion, no learned colour embedding — I hand the centered RGB to KPConv and let the hierarchical strided convolutions figure out how to fuse colour and geometry.

The zero-centering is small but it matters. The baseline's constant-1 input has fixed scale and zero variance, and the first KPConv kernel was implicitly calibrated for that. Raw RGB has mean around 0.5 and per-channel variance, which I found could occasionally produce extreme single-kernel responses on certain fragments and blow up the gradient at the very first layer. Centering halves the worst-case input magnitude and removes the systematic positive bias without losing any colour information.

Two reasons single-level injection is enough. One, KPConv is already a learned convolution — adding two extra input channels just makes the first-layer kernel three-dimensional instead of one. Two, ColorPCR's more elaborate CEFE block fuses colour at multiple levels, but it's overkill if the encoder is already deep enough to propagate the signal. A single-level input injection is a good null-hypothesis before committing to multi-level fusion.

The controls: one new toggle, `use_color`, and one policy knob, `color_source`, which selects between real RGB from the Color3DMatch files, random RGB for architecture sanity checks, and zeros for degradation tests.

DELIVERY *The `color_source: random` trick is a great point to flag — it shows you thought about how to validate the shape flow before the real data landed. The centering paragraph is your robustness story: examiners love hearing that you debugged a real numerical issue rather than assuming the textbook setup just works.*

SLIDE 9 — TASK 2 DEEP-DIVE — RELATIONAL PE

[2:00]

ON SLIDE

- Updated key-property line: 'Verified experimentally: max per-pair difference in the relational tensor under a random rigid transform is under 10^{-3} — dominated by float32 numerics and k-NN tie-breaking.'
- Diagram: pairwise distances + triplet angles $\rightarrow$ MLP $\rightarrow$ projection $\rightarrow$ Q·R relational term in self-attention

SAY

Task 2 is where the more interesting idea is. RegTR's baseline positional encoding is sinusoidal on absolute point coordinates. That means if you rotate or translate the input — the same geometry, same scene, just in a different frame — the positional encoding changes. The self-attention then has to learn to be rotation-invariant implicitly, which is wasteful and fragile.

Instead I build a per-pair relational tensor from two quantities that are, by construction, invariant to rigid transforms. First, pairwise distance embedding: for every pair of points i and j , I compute the Euclidean distance and pass it through a Gaussian Fourier mapping. Second, triplet angle embedding: for each point's k nearest neighbours, I compute angles and embed them the same way. Those feed a small MLP and projection into head space.

In attention, the implementation follows GeoTransformer's Q·R recipe — the relational tensor contracts with the query so the extra term scales with Q, not only a static additive mask. The final projection is zero-initialised so training starts near the baseline and gradually turns the relational pathway on.

The key property: distances and angles are unchanged under any rotation or translation of the cloud. So the relational tensor matches before and after the transform to within about **ten to the minus three**. I verified this experimentally — under a random SE(3) transform, the max per-pair difference is under 10^{-3} , which is dominated by float32 round-off and top-k tie-breaking in the nearest-neighbour search. The test lives in `test_relational_pos_emb.py` and runs in under a second on a CPU.

The source of the idea is GeoTransformer, which used distance-plus-angle structure in correspondence refinement. I'm applying the same structural idea to RegTR's encoder self-attention in Q·R form.

DELIVERY *This is your most technical slide. Slow down on 'invariant to rigid transforms' — let it sink in. If someone's going to ask a hostile question, it's about this slide, so practice saying 'under 10^{-3} ' cleanly. Do NOT say $5e-5$ — the actual number is $1e-3$.*

SLIDE 10 — IMPLEMENTATION & ENGINEERING STATUS**[1:30]****ON SLIDE**

- Config toggles (`use_color`, `use_relational_pos_emb`) for clean ablation
- MinkowskiEngine removed — replaced with pure-PyTorch grid subsampling (`kpconv.py`, `batch_grid_subsampling_kpconv_gpu`)
- Three smoke tests: ME removal, RGB plumbing, rigid-invariance (Q·R wiring)
- Numerical-stability hardening: skips `backward()` on non-finite loss, detaches loss tensors, skips `optimizer.step()` on bad gradients, recovers from CUDA OOM
- `src/data_processing/convert_color3dmatch.py` handles ColorPCR's `.npy` → `.pth` conversion

SAY

Briefly on engineering, because it matters for reproducibility. All changes sit behind config flags — `use_color` and `use_relational_pos_emb` — so reproducing the baseline is literally a one-line config change.

The pipeline builds cleanly on modern CUDA without MinkowskiEngine, which was the original RegTR's external C++ dependency for voxel grid subsampling. I replaced it with a roughly fifteen-line pure-PyTorch scatter-add that's functionally equivalent to Minkowski's unweighted-average quantisation.

Three smoke tests run in CI: one checks that the pipeline imports and runs a forward pass without Minkowski; one checks that the RGB dataloader plumbing is consistent across my three `color_source` policies; and one checks that the relational tensor is rigid-invariant to under 10^{-3} in the Q·R encoder path. A conversion script handles ColorPCR's `.npy` format so the same training loop consumes Color3DMatch without extra tooling.

One point worth flagging: when I started training the colour configuration, I observed occasional non-finite losses propagating through to corrupt the model weights — a known failure mode when finite losses produce non-finite gradients near degenerate configurations of the SVD-based pose head. I hardened the trainer to detect this and skip the offending optimizer step rather than silently overwriting the weights with NaN. The trainer logs which parameter went bad first, which is what told me the instability was localised to the first KPConv layer and motivated the RGB centering on the previous slide.

DELIVERY *The trainer-hardening paragraph is worth saying clearly — it converts a problem you discovered into a piece of engineering rigour. Don't apologise for it; frame it as a positive iteration. Total budget for this slide is 90 seconds.*

SLIDE 11 — PRELIMINARY RESULTS

[2:00]

ON SLIDE

- ⚠️ Replace placeholder chart tonight with real tensorboard exports
- Loss curves, log-y, four lines: baseline, +colour, +relational, REGTR+
- Small table: best val registration recall at current checkpoint (if available)
- Caption: '~5 epochs ≈ 10% of full training — descending and stable, not converged.'

SAY

Preliminary results. Four ablation configurations: the geometry-only baseline, colour-only, relational-PE-only, and the full REGTR+ with both modules. Each run is about five epochs of training, which is roughly ten percent of what a converged run looks like on this baseline.

*I want to be upfront about that — **I'm not claiming state of the art here.*** What the curves do show is two things.

One, adding colour and relational encoding doesn't destabilise training — the loss is monotonically decreasing for all four configs, which is a necessary condition before committing compute to full 40-epoch runs. Two, the ordering of the curves is what we'd expect: both modules independently lower the loss, and the combination is the lowest. That's consistent with the modules being orthogonal contributions rather than redundant.

[If your real table has registration-recall numbers, read the headline number here. If not, say:] I don't have registration-recall numbers on a held-out test split yet — those come after full convergence.

DELIVERY *Examiners will push here. Pre-commit to your shield: 'The curves are descending; the ordering is consistent with the hypothesis; absolute numbers will come from the full 40-epoch runs on the roadmap.' Memorise that sentence.*

SLIDE 12 — QUALITATIVE EXAMPLE

[1:30]

ON SLIDE

- ⚠️ Replace placeholder screenshots tonight (run `python src/demo.py` with pretrained 3DMatch checkpoint)
- Two side-by-side images: BEFORE (misaligned) and AFTER (RegTR aligned)
- Caption: 'Reference: fully-trained baseline RegTR (pretrained, 40 epochs). Our REGTR+ is on this trajectory.'

SAY

A quick qualitative example. This is a pair from the 7-scenes red-kitchen sequence — a standard 3DMatch evaluation pair. On the left, the two clouds in their original viewpoints, misaligned. On the right, the same pair after RegTR — this is the public pretrained checkpoint, forty epochs, running baseline RegTR.

I'm showing the baseline here, not my REGTR+ model, for an honest reason: my REGTR+ runs are still at five epochs and don't look this clean yet. This slide says: this is what fully-converged looks like for the task. My pipeline can produce the same forward pass. Once I finish the full training runs, I'll replace this slide with my own output.

DELIVERY *This is a transparency move. Examiners appreciate a researcher who doesn't overclaim.*

SLIDE 13 — ROADMAP

[1:00]

ON SLIDE

- Task 3 — AOF (dev): MLP-gated attention in last K cross-encoder layers, conditioned on overlap logits
- Task 4 — Uncertainty-Weighted SVD (dev): σ head; $w = \hat{\sigma}/\sigma^2$; heteroscedastic loss per Kendall & Gal 2017
- Full convergence (planned): 40–50 epoch training for all four ablation configs; report RR + RMSE on 3DMatch
- Color3DMatch benchmark (planned): direct comparison vs baseline RegTR and full ColorPCR

SAY

Looking forward. Task 3, active overlap focusing, is in active development — the idea is to use the overlap logits the model already predicts to soft-gate the attention in the last few cross-encoder layers, so capacity isn't wasted on non-overlapping regions.

Task 4, uncertainty-weighted SVD — predict a per-point aleatoric variance alongside the confidence logits, then re-weight the pose solver. This borrows directly from Kendall and Gal's 2017 uncertainty framework.

The two deliverables after that are full convergence runs across all four ablation configurations, and evaluation on the Color3DMatch benchmark against both baseline RegTR and ColorPCR. That benchmark comparison is the headline result I'm aiming for in the final thesis.

DELIVERY *Tight, bullet-pace, energetic. You're on the home stretch.*

SLIDE 14 — SUMMARY

[0:30]

ON SLIDE

- (1) Extended RegTR with colour fusion + relational self-attention (Q·R; hybrid sine + relational on the main Color3DMatch preset) — drop-in, toggleable
- (2) Pipeline runs end-to-end; Color3DMatch four-cell ablation uses matched train_batch_size: 1 and separate YAMLS per cell
- (3) AOF and uncertainty modules in dev; Color3DMatch benchmark is the final deliverable
- Acknowledgements line. 'Thank you — Questions?'

SAY

To close. I extended RegTR with two drop-in, config-toggleable modules: colour fusion at the input, and GeoTransformer-style relational self-attention — on Color3DMatch I use a hybrid that keeps sine PE and adds the relational term.

The pipeline runs end-to-end with four matched ablation configs on Color3DMatch. Overlap-gated attention and uncertainty-weighted SVD are in active development, and the final deliverable is a head-to-head benchmark against ColorPCR on Color3DMatch.

Thank you — happy to take questions.

DELIVERY *Pause after 'happy to take questions.' Don't rush into looking at the floor. Hold eye contact for two seconds. Let the first question come.*

Q & A BANK

Star (★) = very likely. Rehearse those verbatim.

VERY LIKELY — rehearse verbatim

★ Q: Why is colour at the input enough? Why not fuse at multiple KPConv levels like ColorPCR does?

A: Two reasons. First, KPConv is already learned — the first layer is free to combine RGB and XYZ however it likes, and the signal propagates through the hierarchy via the strided downsamples. Second, single-level injection is the null hypothesis I wanted to falsify before committing to a more complex fusion. If the single-level injection plateaus, multi-level is the next step, and the toggle infrastructure makes that easy to prototype.

★ Q: How do you know the improvements come from your modules and not from randomness at 5 epochs?

A: I don't — not rigorously. That's why the full 40-epoch runs are on the roadmap. At 5 epochs the claim is narrower: the pipeline trains stably, the ordering of the four curves is consistent with the hypothesis, and there's no sign the new modules are hurting convergence. The rigorous ablation claim comes from the full runs with multiple seeds.

★ Q: What does $1e-3$ actually mean for rigid invariance? Isn't that large?

A: In absolute terms it's tiny, but it's not zero. Three sources: float32 round-off in the distance computation, floating-point error in the sinusoidal embedding, and occasional tie-breaking in the top-k nearest-neighbour selection where a different k-NN is picked after a rotation. A strict SE(3)-invariance theorem says the bias should be zero; the 10^{-3} number is how far the implementation is from the theorem on an 80-point random cloud. The test catches bugs where the bias drifts by orders of magnitude.

★ Q: You've only implemented two of four tasks. Is that on schedule?

A: Yes. The project proposal allocated the first half for implementation of tasks 1 and 2, the second half for tasks 3 and 4 plus the benchmark. I'm about two weeks ahead on implementation — the MinkowskiEngine removal was a detour I didn't plan for but it means the pipeline now builds cleanly on VastAI without custom CUDA, which was a 2-week de-risking move. The remaining schedule is AOF, then uncertainty-weighted SVD, then full benchmark.

★ Q: Why RegTR as the baseline? Why not start from GeoTransformer or ColorPCR directly?

A: RegTR is the simplest end-to-end model in this space — single forward pass, no RANSAC, no iterative refinement. Starting from a simple baseline means every ablation is attributable. GeoTransformer and ColorPCR have more moving parts, so a single improvement is harder to isolate. Also, RegTR's SVD head gave me a clean place to add the uncertainty weighting in Task 4 — it's a direct insertion point that doesn't exist in ColorPCR.

LIKELY — know the answer

Q: Why zero-center RGB? Wouldn't the network learn to handle the offset on its own?

A: In principle yes — the first KPConv layer's bias term can absorb a constant offset. In practice, I observed during training that occasional fragments produced extreme single-kernel responses with raw $[0,1]$ RGB, which blew up the gradient at `kpf_encoder.encoder_blocks.0.KPConv.weights`. Centering halves the worst-case input magnitude and removes a systematic positive bias the kernel was never initialised for. It's a one-line change with no downside — colour information is preserved exactly — and it eliminated the gradient-spike rate I was seeing. The fact that the trainer's gradient sentinel told me exactly which parameter was the issue is what made the diagnosis tractable.

Q: You said the trainer skips bad gradients. Doesn't that bias training toward easy examples?

A: It would, if the skip rate were high. In practice, after centering, the rate is well under 1% of batches. The fix isn't 'masking the problem' — it's a safety net so that a single numerically degenerate batch doesn't permanently NaN the model weights and waste hours of compute. The actual cure is the centering and gradient clipping; the skip path just stops one bad batch from poisoning the whole run. If the skip rate stayed high, that would be a signal I need a deeper fix — for example a mixed-precision policy or an LR warmup.

Q: Why raw RGB? Why not normalise per-scan for illumination invariance?

A: Per-scan normalisation is the obvious next step, and I didn't want to add two variables at once. Zero-centering establishes that the colour pathway is numerically stable; per-scan illumination normalisation is a clean follow-up. ColorPCR also uses raw RGB without per-scan normalisation, so this choice is consistent with prior art.

Q: Why is relational_sigma_d 1.0 on Color3DMatch but 0.2 in plain 3dmatch.yaml?

A: The distance kernel width has to be matched to the coordinate spacing of the tokens feeding self-attention. After KPConv's strided hierarchy, bottleneck points sit roughly 0.2 m apart on indoor scans. With $\text{sigma_d} = 0.2$, the Gaussian factor in the relational embedding was effectively zero for almost all pairs — only nearest neighbours mattered. Bumping to 1.0 on the Color3DMatch relational configs makes the distance channel informative across more of the attention matrix. Plain 3dmatch.yaml still lists 0.2 as the legacy default for configs that don't use relational PE at all or for smaller-scale ablations — always read the YAML you're actually running.

Q: Why triplet angles with k=3 neighbours? Why not all triplets?

A: Cost. All-triplets is $O(N^3)$ in the attention tensor. 3-NN is $O(N^2 \times K)$ which is tractable for N up to a few thousand. Empirically 3 is what GeoTransformer used, so I inherited it.

Q: What's color_source: random for?

A: It's a shape-validation tool. When I was plumbing the colour path but didn't yet have Color3DMatch data, I could run training with random RGB — the geometry is still learned normally, and the extra channels get routed through the encoder. It confirms every shape and tensor flow works without needing real colour data. It also serves as a negative control: if `color_source: random` gives similar results to real colour, that's evidence the colour isn't doing useful work.

Q: You removed MinkowskiEngine — is your grid subsampling a bottleneck?

A: On the workloads I've profiled — batch size 1 on Color3DMatch ablations, two point clouds up to ~20k points — no. The pure-PyTorch scatter-add runs at similar throughput per step. On very large clouds there may be a crossover point, but I haven't hit it. The tradeoff was build complexity versus speed, and build complexity was killing iteration velocity on VastAI.

Q: What's your batch size? Can you scale to multi-GPU?

A: On Color3DMatch ablations I standardised on `train_batch_size: 1` on a VastAI $2 \times \sim 16$ GB pod so every ablation cell sees the same gradient-noise profile — that was a deliberate fairness fix once relational memory forced batch 1 for the full model. Plain `3dmatch.yaml` on the original pipeline may still use a larger batch where memory allows. Multi-GPU isn't wired in yet — DDP with variable point counts needs careful collation; if wall-clock becomes the bottleneck I'll add it.

Q: How does your test suite compare to the original RegTR's?

A: The original repo has no automated tests — everything was verified manually via the `demo.py` script. I added three CI-runnable smoke tests specifically to catch regressions in the parts I touched: the Minkowski rip-out, the RGB plumbing, and the relational-PE invariance. They're not comprehensive unit tests but they catch 100% of the regressions I've introduced during development so far.

HARD — you should still have answers

Q: Your relational PE is SE(3)-invariant, but the final registration is not — the whole point is to recover a rigid transform. Isn't SE(3)-invariant attention the wrong inductive bias?

A: Good question. The invariance is a property of the relational structure pathway in self-attention, not the whole pipeline — and on my default Color3DMatch preset I still keep absolute sine PE alongside relational (hybrid), so I'm not claiming the full stack is strictly invariant. Self-attention builds within-cloud features; the relational piece is designed so that that contribution doesn't care about global frame. The rigid transform is recovered in cross-attention and SVD, which are not invariant. So the inductive bias is at the right level: stabilise geometric structure, learn alignment elsewhere.

Q: How are you handling the case where one cloud has colour and the other doesn't, or they come from different sensors with different colour calibration?

A: Not handled in the current pipeline — both clouds must have RGB. The `color_source: zeros` policy is a degenerate case that tests the codepath but doesn't model a realistic multi-sensor scenario. Cross-sensor colour is a thesis-scope-adjacent problem; I'd flag it as future work.

Q: You're comparing to ColorPCR on Color3DMatch, but ColorPCR uses a different correspondence formulation — is that a fair comparison?

A: It's fair at the task level — same input, same output metric (registration recall, RMSE, chamfer). It's less fair architecturally, but that's true of any cross-architecture benchmark. The honest framing is: 'REGTR+ is a simpler model that recovers a large fraction of ColorPCR's gains with a fraction of the components.' I don't expect to beat ColorPCR; I expect to be competitive at lower cost.

Q: Why not use a learned positional encoding conditioned on the point cloud — like in RoPE or ALiBi — instead of a handcrafted distance-angle bias?

A: Learned PE would be a reasonable alternative. The reason I went with distance-angle structure is that the relational tensor is rigid-invariant by construction, rather than something the network must learn from scratch. For rigid registration that's a strong inductive bias on the relational pathway. If a learned PE wins empirically, the design is modular enough to try it.

Q: How do you handle the case where test_rigid_invariance passes at 1e-3 on an 80-point cloud but fails on a 20,000-point real cloud?

A: Two things. One, the invariance is a per-pair property — it's independent of cloud size, so the per-pair numerics should be the same. Two, I haven't tested on 20k points; that's a valid gap. I'll add a larger-scale test on real 3DMatch data before the final submission.

DELIVERY & RECOVERY

VOICE

Stand at roughly the same distance from the mic the whole time. Don't drift. When showing the architecture diagram (slide 7), slow down — your brain wants to race; the audience's eyes can't keep up. Breathe audibly between sections. A one-second pause after 'the key property' is worth ten seconds of fluent talking.

BODY

Don't turn your back to the audience to read your own slides. You should know them cold enough that you face out. If you can, use a remote clicker. Staying out from behind the laptop is worth 10% of your perceived confidence.

SLIDES

Dark mode / dim the room if possible — the figures on slides 11 and 12 will pop more. Have slide 7 (architecture) printed as a handout for your supervisor if they like that. It's the only slide worth the paper.

WHEN SOMETHING GOES WRONG

If a demo image doesn't load: "That's the registration output — I can show it after if you'd like." Move on.

If you forget a number: "Let me come back to that — off the top I don't want to misquote." Then actually come back to it.

If you blank on a word: stop, breathe, drink water. Silence reads as composure. Filler words ('um, like, you know') read as panic.

THE LINES TO SAY EXACTLY RIGHT

- "Throw this signal away entirely." (slide 3 — your motivation lands)
- "Drop-in, config-toggleable module." (slide 6 — your methodological maturity)
- "Invariant to rigid transforms... under 10^{-3} ." (slide 9 — NOT 5×10^{-5})
- "Per-point RGB, zero-centered." (slide 8 — NO HSV)
- "I'm not claiming state of the art here." (slide 11 — disarms the hostile question)
- "Thank you — happy to take questions." (slide 14 — then PAUSE)